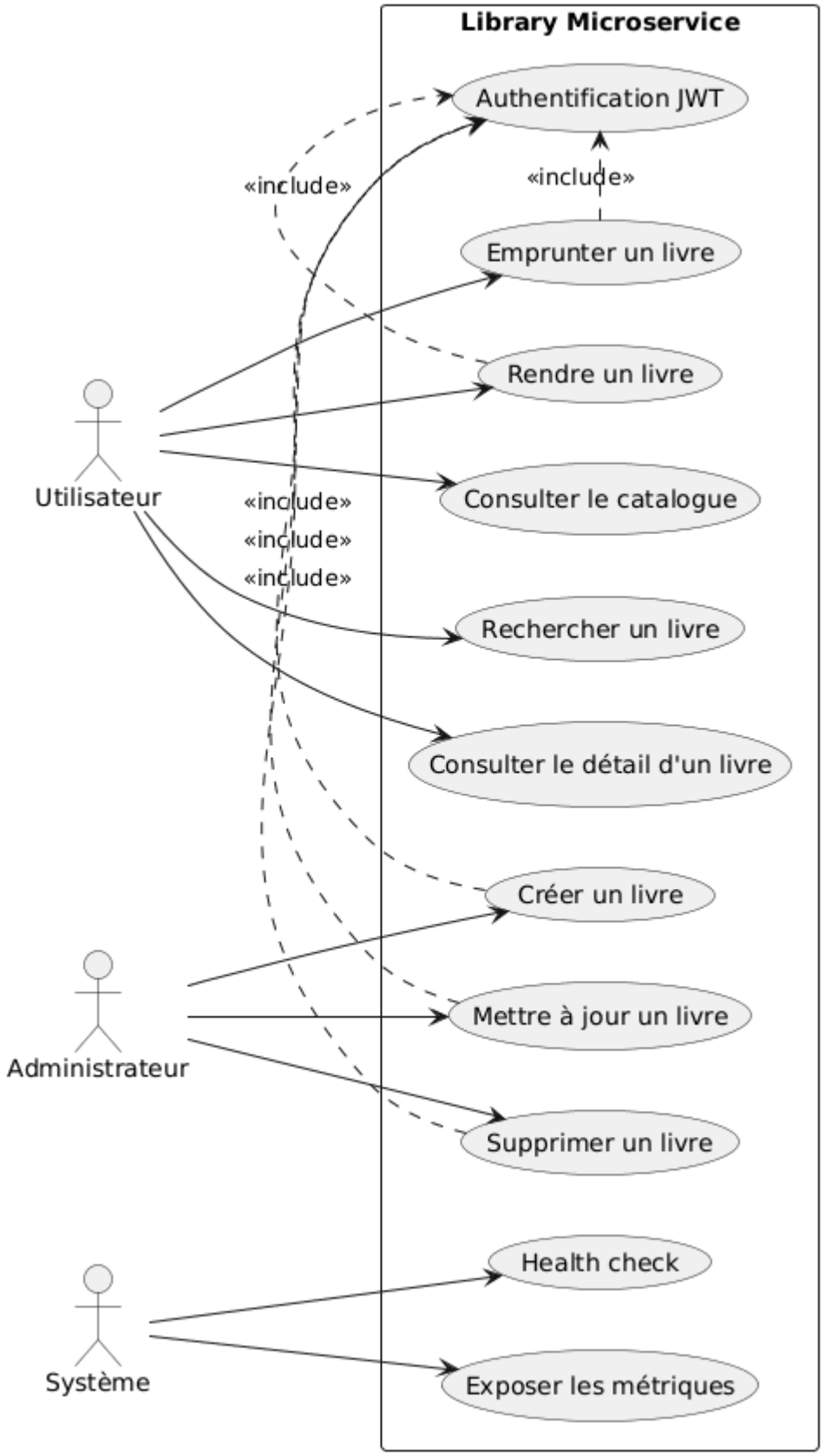
Diagramme de cas d'utilisation (Use Case) :



Le diagramme de cas d'utilisation présente les interactions entre les acteurs externes et le microservice de gestion de bibliothèque. Trois acteurs sont identifiés : l'Utilisateur, l'Administrateur et le Système.

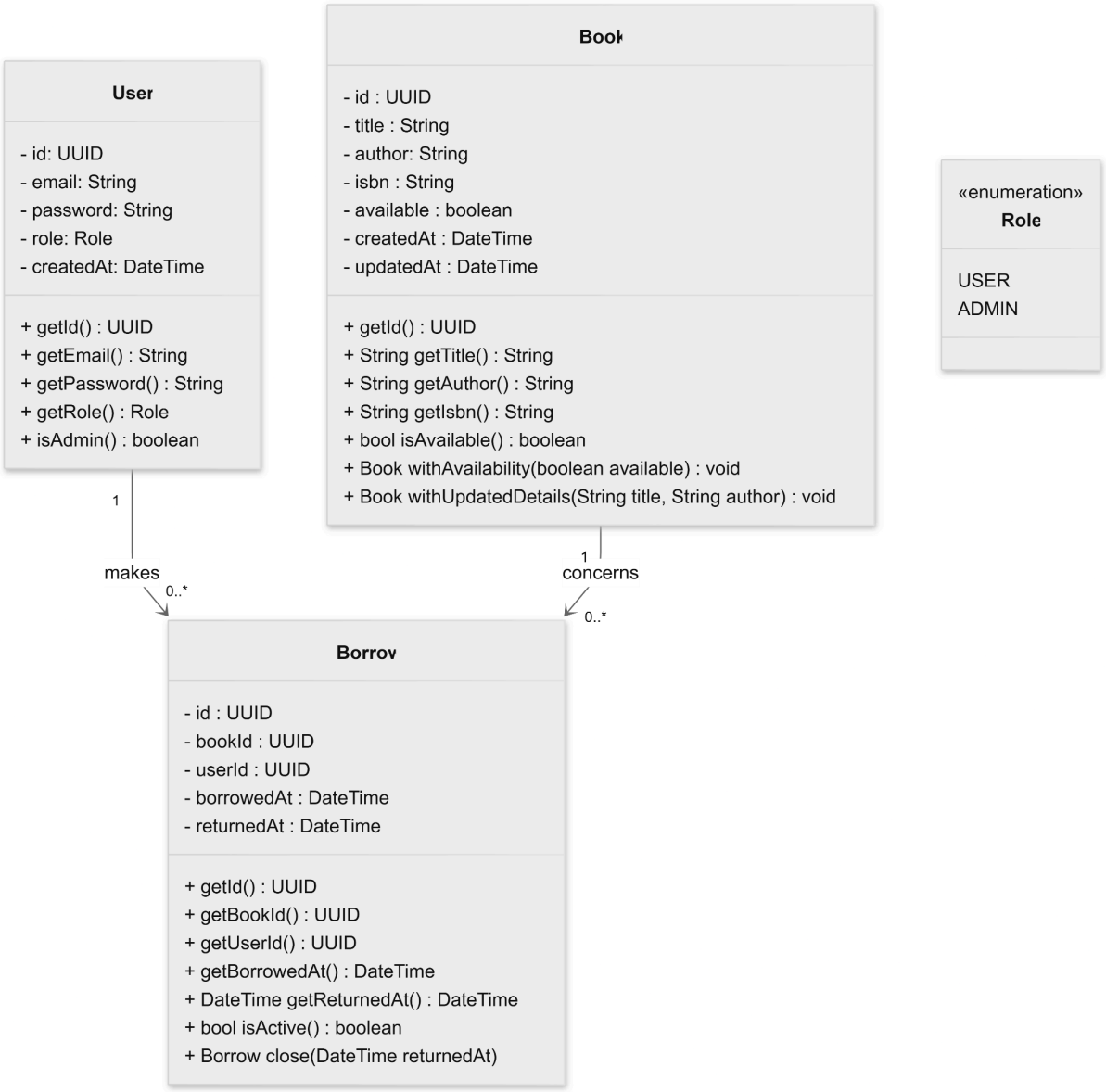
L'Utilisateur peut consulter le catalogue, rechercher un livre, consulter le détail d'un livre, emprunter un livre et rendre un livre. Les opérations d'emprunt et de retour incluent une authentification JWT préalable.

L'Administrateur dispose de droits supplémentaires lui permettant de créer, modifier et supprimer des livres.

Le Système interagit avec le microservice pour effectuer un health check et exposer les métriques à des fins de monitoring.

Ce diagramme met en évidence la séparation des responsabilités et le contrôle d'accès basé sur les rôles.

Diagramme de classes :



Le diagramme de classes modélise les entités métier principales : User, Book et Borrow, ainsi que l'énumération Role.

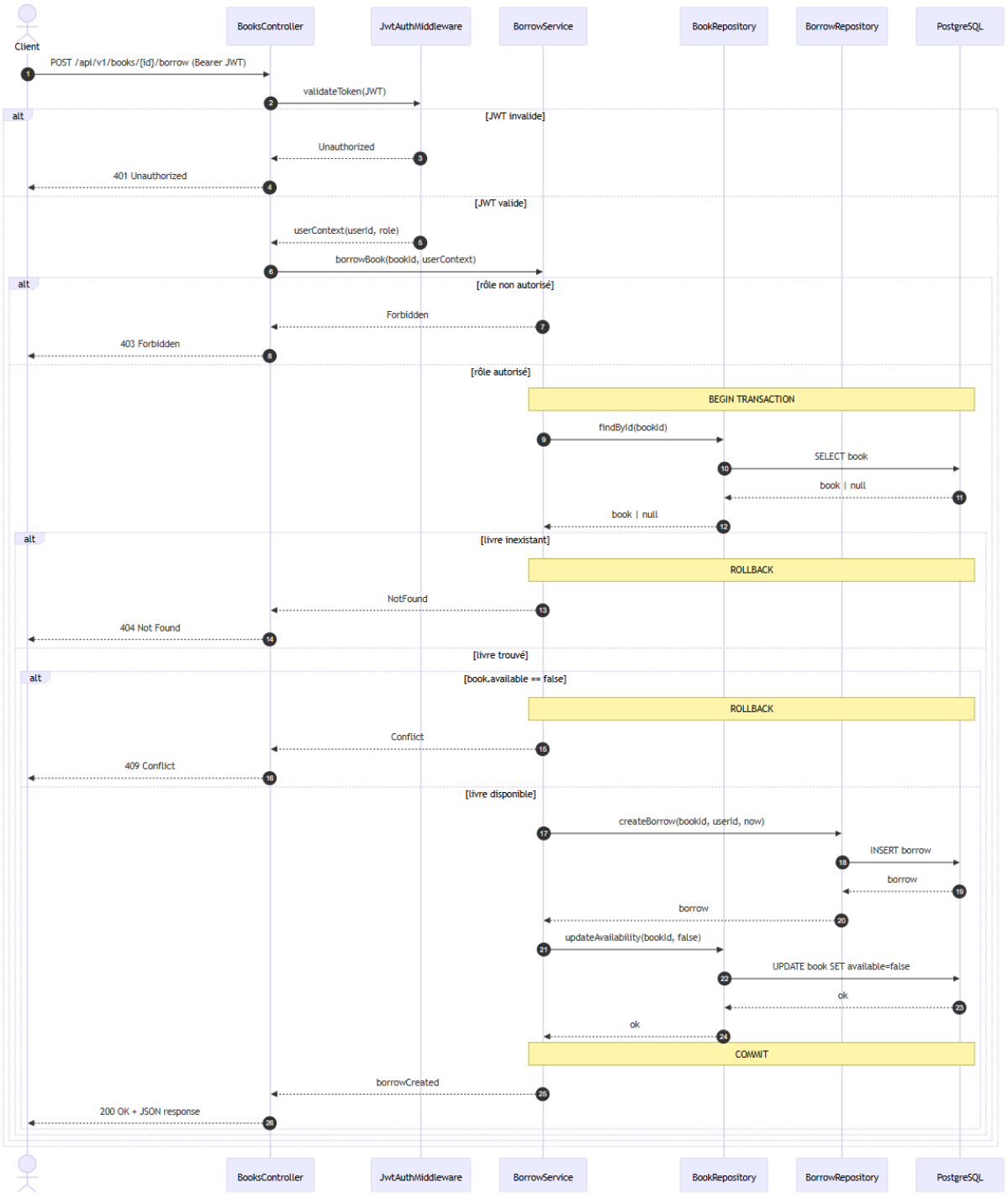
La classe Book contient les informations bibliographiques (titre, auteur, ISBN) et un attribut de disponibilité. Elle expose des méthodes permettant de modifier son état, notamment la mise à jour des informations et la gestion de la disponibilité.

La classe User contient les informations d'authentification et un rôle (USER ou ADMIN).

La classe Borrow représente l'association entre un utilisateur et un livre, avec les dates d'emprunt et de retour. Elle permet de déterminer si un emprunt est actif et de clôturer un emprunt.

Les relations indiquent qu'un utilisateur peut effectuer plusieurs emprunts et qu'un livre peut être concerné par plusieurs emprunts au cours du temps.

<u>Diagramme de Séquence</u> :



Le diagramme de séquence illustre le scénario complet d'un emprunt de livre via l'endpoint `POST /api/v1/books/{id}/borrow`.

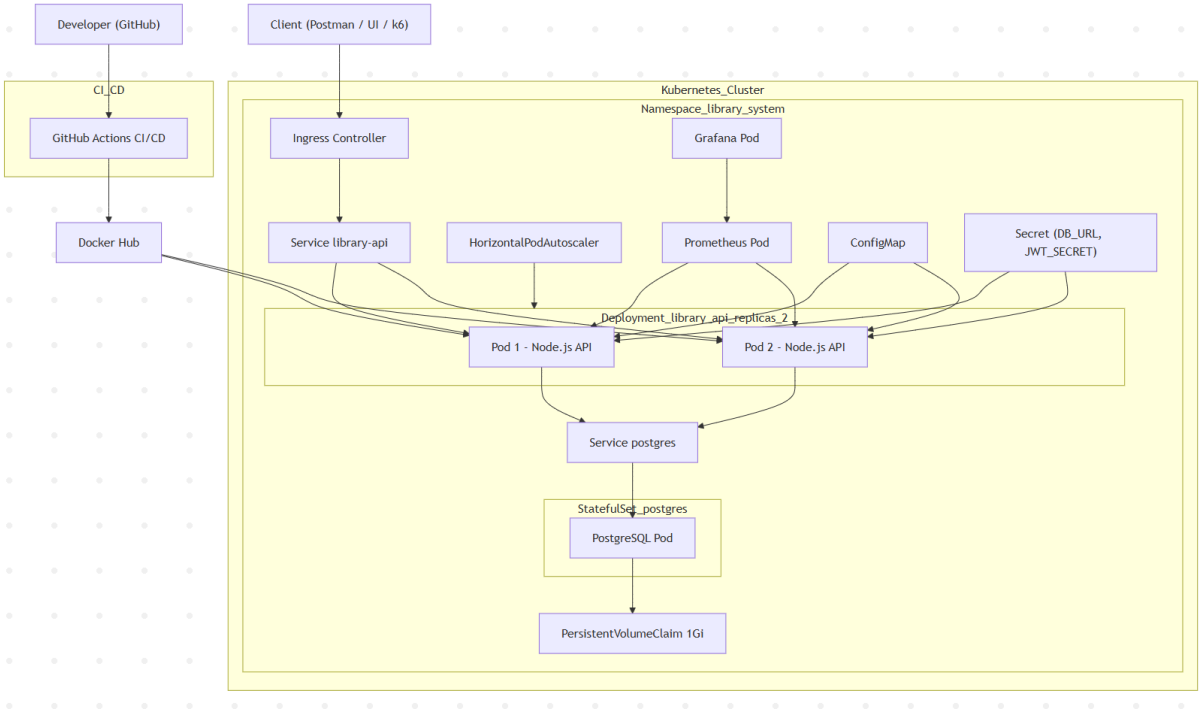
Le flux commence par une requête du client vers le contrôleur, qui valide le token JWT via le middleware d'authentification. En cas d'échec, une réponse 401 est retournée.

Si l'utilisateur est autorisé, le service métier vérifie l'existence du livre et sa disponibilité. Plusieurs cas alternatifs sont représentés : livre inexistant (404), livre indisponible (409), ou utilisateur non autorisé (403).

En cas de succès, une transaction est ouverte : un enregistrement d'emprunt est créé et la disponibilité du livre est mise à jour. La transaction est ensuite validée (commit), et une réponse 200 OK est retournée.

Ce diagramme montre clairement la gestion des erreurs, la logique métier et la transactionnalité.

<u>Diagramme de Déploiement</u> :



Le diagramme de déploiement décrit l'architecture d'exécution dans un cluster Kubernetes.

Le microservice est déployé dans un namespace dédié et exécuté sous forme de pods répliqués pour assurer la haute disponibilité. Il est exposé via un Ingress Controller.

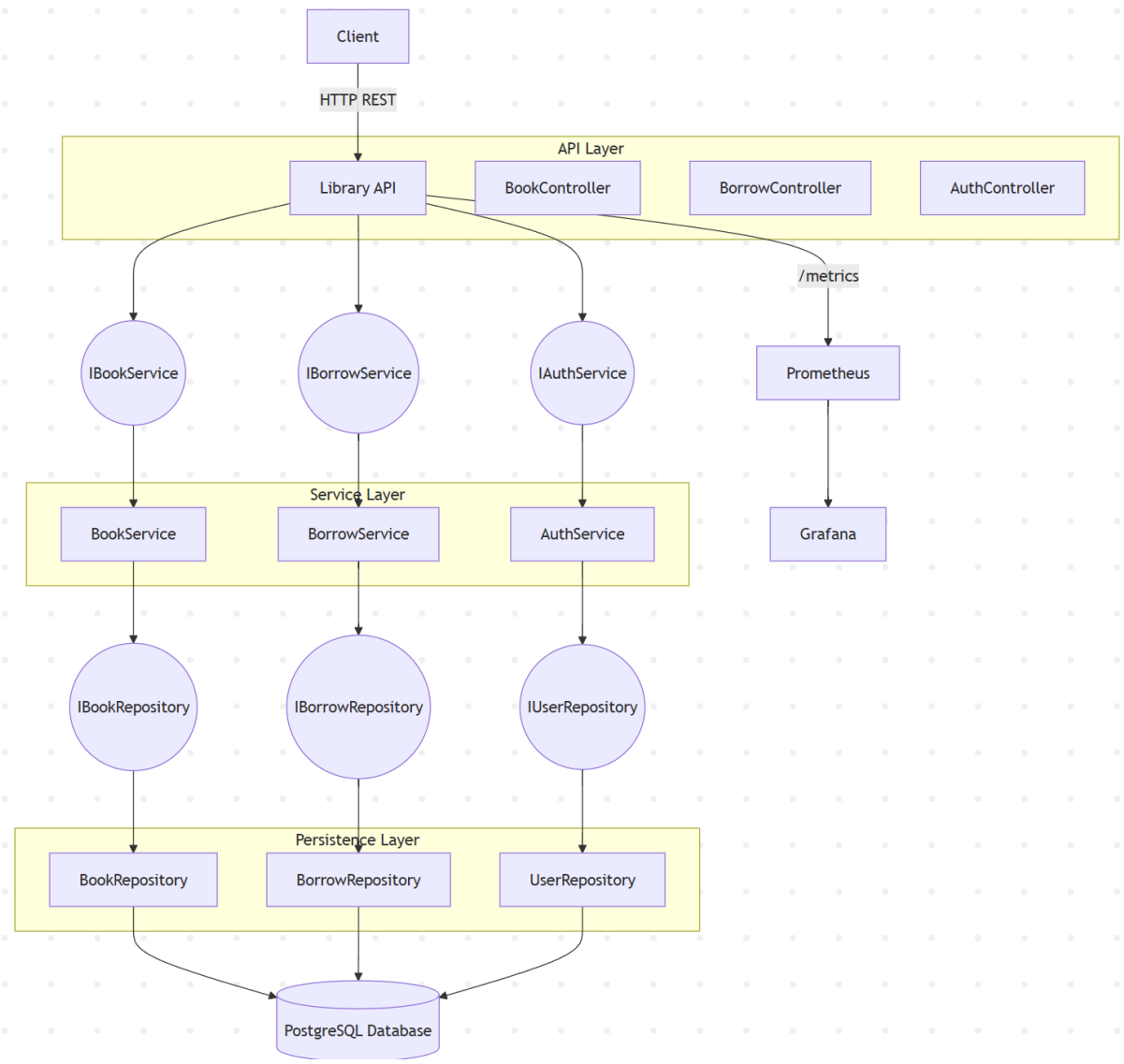
La base de données PostgreSQL est déployée avec un PersistentVolumeClaim pour garantir la persistance des données.

Prometheus collecte les métriques exposées par l'API, et Grafana permet leur visualisation.

Le pipeline CI/CD (GitHub Actions) construit l'image Docker, la publie dans un registre, puis déclenche le déploiement dans le cluster.

Ce diagramme met en évidence la séparation entre l'environnement de développement, le pipeline d'intégration continue et l'environnement d'exécution.

<u>Diagramme de Composant</u> :



Le diagramme de composants illustre l'architecture en couches du microservice.

La couche API contient les contrôleurs (`BookController`, `BorrowController`, `AuthController`) et expose les endpoints REST.

La couche Service implémente la logique métier via `BookService`, `BorrowService` et `AuthService`. Elle dépend d'interfaces de repository pour accéder aux données.

La couche Persistence implémente les interfaces `IBookRepository`, `IBorrowRepository` et `IUserRepository`, responsables des interactions avec la base PostgreSQL.

Le diagramme montre l'inversion de dépendance : les services dépendent d'abstractions et non d'implémentations concrètes.

Enfin, le composant Prometheus consomme l'endpoint `/metrics` exposé par l'API, et Grafana visualise les données collectées.